HearthSync+ : Product Requirements Document (PRD)

Autonomous Multimodal Household & Smart-Kitchen Commerce Concierge on Alexa+ via Streamable HTTP MCP & Agent Skills

Document Property	Specification & Target Track Details	Hackathon Deadlines & Prize Pool
Hackathon & Primary Track	Build, Ship, Shape: Amazon Developer Hackathon — **Alexa+ Primary Track**	**Submission Deadline:** Oct 23, 2026 @ 12:00 PM PDT (Official Rules)
Core Standards Implemented	MCP Streamable HTTP (2025-11-25 & 2026-07-28) + Agent Skills Open Standard (SKILL.md) + MCP Apps Extension	**Alexa+ Track Prizes:** $25,000 (1st), $15,000 (2nd), $4,000 (3rd) + up to $15,000 AWS Credits & Amazon 1:1
Layered Mini-Challenges	**AWS Builder** (Amazon Bedrock AgentCore + Strands SDK) OR **Open Source** + **Friction Logs**	**Mini-Challenge Prize:** $5,000 Cash + $5,000 AWS Credits; **Friction Log Bonus:** Up to +10% judging score

1. Executive Summary: Bridging Agent Skills and Streamable HTTP MCP for Alexa+

HearthSync+ is an autonomous, multimodal household and kitchen commerce orchestrator designed for ambient screen-equipped devices such as the Amazon Echo Show 21 and Echo Show 15. Built for the Build, Ship, Shape: Amazon Developer Hackathon, HearthSync+ directly fulfills the Alexa+ track mandate by combining a self-hosted **Model Context Protocol (MCP) server over Streamable HTTP** with modular, portable **Agent Skills (`SKILL.md`)** and a rich **Alexa+ Multimodal Web Simulator**.

As confirmed in the Hackathon FAQs and clarified by Alexa Chief Evangelist Emerson Sklar, Amazon's internal Alexa+ MCP Toolkit, `@alexa-ai/cli`, and partner Web Simulator are gated in private preview and inaccessible to hackathon entrants. To win the Alexa+ track, submissions must avoid single-turn Q&A bots or thin API wrappers and instead demonstrate **multi-service autonomous orchestration**, **cross-session state persistence**, **transactional purchasing**, and **multimodal visual UI cards/carousels** via a web application acting as a real Streamable HTTP MCP client.

2. Problem Statement: Dual Consumer & Developer Bottlenecks Leave Smart Displays Underutilized

2.1 The Consumer Problem: The Smart-Kitchen Action Gap

Families place Alexa+ smart displays in the kitchen as household command centers, yet daily coordination across pantry inventory, nutrition, meal planning, and grocery checkout fails in three predictable ways:

- **Stateless Single-Turn Amnesia:** Conventional voice skills forget household dietary rules (such as vegetarian, nut-free, or high-protein macro goals), existing pantry stock, and weekly grocery budget caps between conversations.
- **Voice-Only Cognitive Overload in Commerce:** Auditing 12 missing recipe ingredients, comparing unit prices, or approving brand substitutions via voice audio alone is slow and error-prone. Users require synchronized voice summaries paired with touch-interactive visual cards and carousels (Echo Show Visual Shopping).
- **Disconnected Silos Between Planning and Checkout:** Discovering a recipe for expiring spinach does not automatically reconcile existing spice-rack inventory, compute lowest-cost grocery substitutions, stage a checkout cart, and set multi-stage cooking timers on the display.

2.2 The Developer Problem: Monolithic Prompts, Legacy Transports, and Gated Tooling

- **Prompt Bloat vs. Progressive Disclosure:** Hardcoding complex domain workflows (nutritional math, cart substitution rules, checkout safety checks) into a single monolithic system prompt consumes thousands of tokens on every turn. Adopting the Agent Skills Open Standard solves this via 3-stage progressive disclosure (~100-token metadata discovery, <5,000-token `SKILL.md` activation, and on-demand script/reference execution).
- **Deprecated HTTP+SSE vs. Streamable HTTP:** Most community MCP servers still use the deprecated `2024-11-05` dual-endpoint `HTTP+SSE` transport or local `stdio` pipes. However, the Alexa+ MCP QuickStart Guide mandates **Streamable HTTP** (minimum spec 2025-11-25, forward-compatible with 2026-07-28), strict `<500ms` round-trip latency, and OAuth 2.1 with PKCE (`S256`).
- **Lack of an Open Alexa+ MCP Testing Harness:** Because the partner simulator is gated (Hackathon FAQs), developers need an open-source web simulator and automated `addon.json` manifest generator to validate Alexa+ MCP compliance locally.

3. Judging Criteria Mapping: Engineering specifically for "Creative" Differentiators

The Build, Ship, Shape Official Rules explicitly contrast low-scoring "Obvious" submissions against high-scoring "Creative" submissions across four equally weighted pillars (25% each) plus up to a 10% Friction Log bonus:

Judging Criterion	"Obvious" Submission (Low Score per Rules)	HearthSync+ Architecture (High-Scoring "Creative" Execution)
Tech Implementation (25%)	Basic MCP wrapper around a single REST API; README-only mention of MCP/Skills	Self-hosted Streamable HTTP MCP server (supporting both `2025-11-25` sessions and `2026-07-28` header mirroring), 3 spec-validated Agent Skills, OAuth 2.1 PKCE discovery, and `<200ms` tool latency.
Design & UX (25%)	Text-only chatbot window with no smart-display ergonomics	Multimodal Echo Show 21 simulator with Web Speech voice I/O, interactive MCP Apps (`@modelcontextprotocol/ext-apps`) cart carousels, and a live JSON-RPC/SSE Protocol Inspector drawer for judges.

Judging Criterion	"Obvious" Submission (Low Score per Rules)	HearthSync+ Architecture (High-Scoring "Creative" Execution)
Potential Impact (25%)	Niche demo utility with no path to production or developer reuse	Cuts household food waste and grocery spend for consumers while shipping a reusable open-source `alexa-addon-packager` Agent Skill for the Alexa+ developer ecosystem.
Quality of Idea (25%)	Single-turn Q&A or latent LLM recipe generation without stateful tools	Autonomous multi-service workflow (Pantry Audit → Macro Meal Plan → Multi-Store Cart Diff & Price Swap → Guardrailed Checkout → Synchronized Kitchen Timers) with persistent cross-session memory.
Friction Log (+10% Bonus)	Left blank or generic feedback	Structured engineering friction logs analyzing `2025-11-25` vs. `2026-07-28` transport diffs, `401 Unauthorized` without `WWW-Authenticate` per Alexa+ QuickStart, and SSE proxy buffering.

4. System Architecture & Protocol Specification Compliance

HearthSync+ implements the complete 5-component pipeline defined in the Alexa+ MCP Toolkit Overview (MCP Server, Alexa+ MCP Add-on Package, Add-on Registry & Skill Discovery Layer, AI Reasoning & Skill Loader, and Multimodal Orchestrator):

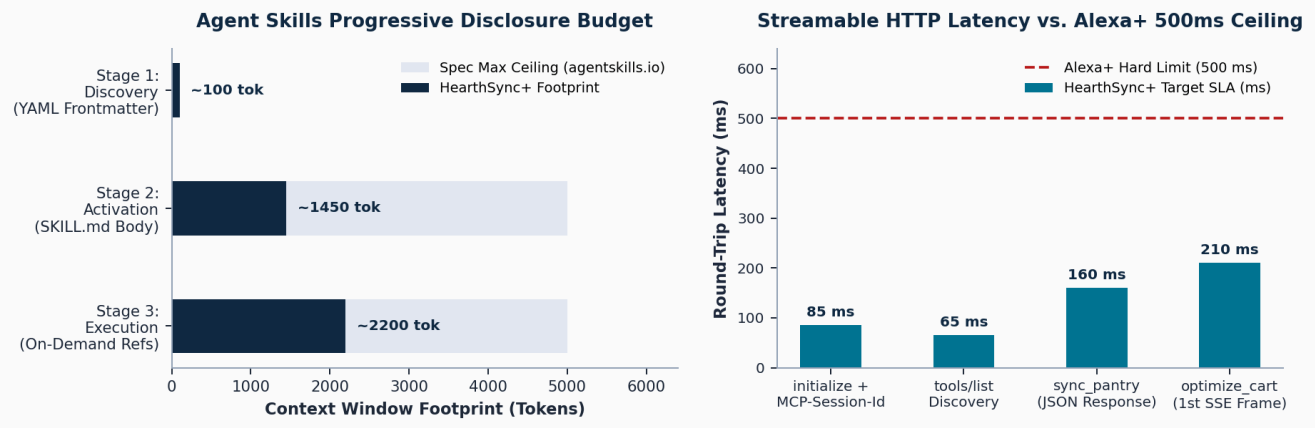


Figure 1: HearthSync+ engineering budgets grounded in the Agent Skills Specification (~100-token metadata discovery vs. <5,000-token body activation at agentskills.io) and the Alexa+ MCP QuickStart latency requirement (<500ms round-trip query response).

4.1 Streamable HTTP MCP Server Specification (`hearthsync-mcp-server`)

To ensure compatibility with both the Alexa+ MCP Toolkit baseline (2025-11-25) and the latest MCP 2026-07-28 Streamable HTTP specification, the server exposes a unified `/mcp` endpoint implementing the following transport mechanics:

Protocol Layer	Specification Requirement (2025-11-25 & 2026-07-28)	HearthSync+ Implementation Detail
Single Endpoint & Content Types	`POST /mcp` accepts JSON-RPC 2.0 messages with `Accept: application/json, text/event-stream`. Notifications return `202 Accepted`.	Fast reads (`sync_pantry_inventory`) return `Content-Type: application/json` in `<160ms`; multi-step cart builds open an SSE stream (`text/event-stream`, `X-Accel-Buffering: no`) emitting `notifications/progress` before the final result.
Session & Header Mirroring	`2025-11-25` uses `MCP-Session-Id` & `MCP-Protocol-Version`; `2026-07-28` mirrors `Mcp-Method`, `Mcp-Name`, and `x-mcp-header` parameters.	Dual-mode middleware issues cryptographic `MCP-Session-Id` headers on `initialize` while validating `Mcp-Method`, `Mcp-Name`, and `Mcp-Param-HouseholdId` (returning `-32020 HeaderMismatch` on body/header discrepancy).
DNS Rebinding & OAuth 2.1 PKCE	Strict `Origin` header validation (`403 Forbidden` on invalid origin). Alexa+ QuickStart requires RFC 9728 PRM & OAuth 2.1 PKCE (S256).	Validates `Origin` on every request; serves `/.well-known/oauth-protected-resource` and `/.well-known/oauth-authorization-server` with `code_challenge_methods_supported: ["S256"]` and `401 Unauthorized` (omitting `WWW-Authenticate` per Alexa+ spec).

4.2 Exposed MCP Primitives: Tools, Stateful Resources, and MCP Apps UI

Primitive	Identifier / URI	Input Parameters (`x-mcp-header` annotated)	Stateful Behavior & Multimodal Output
Tool	`sync_pantry_inventory`	`householdId` (`x-mcp-header: HouseholdId`), `action` (`list|upsert|consume`), `items[]`	Reads/updates persistent pantry DB; computes urgency score for items expiring within 48 hours (e.g., spinach, paneer, Greek yogurt).
Tool	`plan_expiry_meals`	`householdId`, `dietaryTags[]`, `maxPrepMinutes`, `servings`	Matches expiring inventory against household allergies and macro targets; returns 3 structured meal cards with per-serving protein/cost metrics.

Primitive	Identifier / URI	Input Parameters (`x-mcp-header` annotated)	Stateful Behavior & Multimodal Output
Tool	`optimize_smart_cart`	`householdId`, `recipeIds[]`, `budgetCapUsd`, `preferOrganic`	Streams SSE `notifications/progress` across 3 stages: (1) Pantry diffing, (2) Retailer unit-price comparison & auto-substitution, (3) Attaching `ui://hearth/cart-carousel` MCP App.
Tool	`authorize_checkout`	`householdId`, `cartId`, `userConfirmationToken`	Enforces explicit user confirmation token and budget ceiling; commits simulated order, deducts budget, and updates pantry restock ETA.
Tool	`stage_kitchen_timers`	`householdId`, `recipeId`, `stepTimers[]`	Pushes multi-burner named timers and step-by-step voice readouts to the Echo Show ambient bar.
Resource	`hearth://household/memory`	URI read via `resources/read`	Cross-session JSON state persisted in SQLite / Amazon Bedrock AgentCore Memory (allergies, budget ceiling, past meal ratings, active cart).
Resource (MCP App)	`ui://hearth/cart-carousel`	URI read via `@modelcontextprotocol/ext-apps`	Interactive HTML/JS UI bundle rendered inside the Echo Show conversation view allowing 1-tap ingredient swaps and quantity toggles.

5. Modular Agent Skills Design (`.agents/skills/` Open Standard)

Following the Agent Skills Specification, HearthSync+ packages procedural domain intelligence into three self-contained skill directories validated via `skills-ref validate`. Each skill keeps its `SKILL.md` under 500 lines and separates heavy lookup tables and deterministic logic into `references/` and `scripts/`:

Skill Path & Frontmatter `name`	Discovery `description` Trigger (~100 Tokens)	Activated Procedural Workflow & Bundled Files
`.agents/skills/pantry-meal-architect/` `name: pantry-meal-architect`	Plans waste-reducing household meals by cross-referencing expiring pantry inventory, family allergies, and macro targets. Use when asked what to cook, meal planning, or expiring groceries.	1. Read `hearth://household/memory` and call `sync_pantry_inventory`. 2. Consult `references/macro-substitution-matrix.md` for allergen-safe high-protein swaps. 3. Constrain spoken Alexa response to ≤2 warm sentences while rendering visual cards on screen.
`.agents/skills/smart-cart-checkout/` `name: smart-cart-checkout`	Builds budget-optimized grocery carts, executes unit-price ingredient substitutions, and manages safe transactional checkout. Use when adding ingredients to cart, comparing prices, or checking out.	1. Diff recipe ingredients against current pantry stock. 2. Execute `scripts/optimize_unit_price.py` to replace overpriced items under `budgetCapUsd`. 3. Present interactive `ui://hearth/cart-carousel` and require explicit voice/touch confirmation before calling `authorize_checkout`.
`.agents/skills/alexa-addon-packager/` `name: alexa-addon-packager`	Introspects a live Streamable HTTP MCP server and scaffolds a compliant Alexa+ addon-package/addon.json manifest. Use when packaging, validating, or deploying an MCP server for Alexa+.	1. Send JSON-RPC `initialize` and `tools/list` to the target HTTPS MCP URL. 2. Run `scripts/scaffold_addon_manifest.py` against `references/alexa-addon-schema.json` (Alexa+ QuickStart Schema) to generate icons, carousel config, and `examplePhrases`.

6. Simulated Alexa+ Multimodal Web App Experience & User Flows

The web application simulates an Echo Show 21 wall-mounted smart display on the left (70% viewport) paired with a collapsible **Judge & Developer Protocol Inspector** on the right (30% viewport):

- **Echo Show 21 Ambient Surface (Left Pane):** Features an animated Alexa+ cyan-blue listening orb (Web Speech API STT + neural TTS), persistent household memory chips (*"Vegetarian • Nut-Free • $65/wk Budget • 3 Expiring Items"*), touch-scrollable recipe cards, an embedded MCP App cart drawer for 1-tap brand substitutions, and live kitchen timers.
- **Live MCP & Agent Skills Wire Inspector (Right Pane):** Displays real-time HTTP POST traffic to `/mcp`, showing request headers (`MCP-Protocol-Version: 2025-11-25`, `MCP-Session-Id`, `Mcp-Method`, `Mcp-Name`), live SSE `notifications/progress` frames, round-trip latency badges (verifying `<500ms` compliance), and the 3-stage Agent Skill progressive disclosure state (Discovery → Activation → Execution).

3-Minute Hackathon Demo Video Storyboard (Strict <3:00 Limit per Rules)

Timestamp	Demo Scene & User Utterance / Action	Under-the-Hood Protocol & Judging Proof Shown on Screen
0:00 – 0:30	**Hook & Architecture Overview:** Show Echo Show 21 simulator with 3 expiring items (spinach, paneer, Greek yogurt) and persistent household profile loaded from a previous session.	Inspector highlights `POST /mcp` (`initialize` → `MCP-Session-Id` issued in `78ms`) and Stage 1 Skill Discovery (~100 tokens/skill).
0:30 – 1:20	**Expiring Pantry to Multimodal Meal Plan:** User says, *"Alexa, plan two high-protein dinners using our expiring spinach and paneer under $25."*	Orchestrator activates `pantry-meal-architect/SKILL.md`, calls `sync_pantry_inventory` + `plan_expiry_meals`, speaks a 2-sentence summary, and renders interactive recipe cards.
1:20 – 2:15	**Autonomous Cart Optimization & MCP App Checkout:** User taps *"Stage Smart Cart"*. User swaps an ingredient inside the inline MCP App widget and says, *"Confirm checkout and start prep timers."*	Server streams SSE `notifications/progress` as `smart-cart-checkout` swaps pine nuts for pumpkin seeds (saving $6.40), mounts `ui://hearth/cart-carousel`, executes `authorize_checkout`, and stages kitchen timers.
2:15 – 2:55	**Developer Bonus — `alexa-addon-packager` Skill:** Click *"Export Alexa+ Add-on Package"* in the Inspector drawer.	Runs `alexa-addon-packager` against live `/mcp` endpoint, generating a schema-valid `addon-package/addon.json` ready for `alexa-ai deploy`.

7. 7-Day Sprint Plan, Repository Compliance & Bonus Strategy (Oct 16 – Oct 23, 2026)

Phase & Dates	Core Engineering Deliverables	Hackathon Rule & Bonus Verification
Sprint 1 (Oct 16–17) MCP Server & Auth	Build TypeScript/FastAPI `/mcp` server supporting Streamable HTTP (`application/json` & `text/event-stream`), `Origin` check, `MCP-Session-Id`, `Mcp-*` header mirroring, and OAuth 2.1 `.well-known` endpoints.	Verify runtime code imports & invokes MCP SDK over Streamable HTTP (mandatory per Section 4 Submission Rules) with `<200ms` latency.

Phase & Dates	Core Engineering Deliverables	Hackathon Rule & Bonus Verification
Sprint 2 (Oct 18–19) Agent Skills & AWS	Write `pantry-meal-architect`, `smart-cart-checkout`, and `alexa-addon-packager` in `.agents/skills/`. Connect Amazon Bedrock + Strands Agents SDK (plus local API-key fallback for judges).	Qualifies project for the **$5,000 AWS Builder Mini-Challenge**; validate all skills via `skills-ref validate`.
Sprint 3 (Oct 20–21) Echo Show Web App	Build Next.js Echo Show 21 simulator, Web Speech voice I/O, `@modelcontextprotocol/ext-apps` interactive cart UI resource, and live JSON-RPC/SSE Protocol Inspector drawer.	Demonstrates real browser MCP client calling `initialize`, `tools/list`, and `tools/call` per Hackathon FAQ guidance.
Sprint 4 (Oct 22–23) Open Source, Video & Submit	Publish standalone `alexa-addon-packager` skill repo with MIT License (or submit PR to an MCP repo) for the **$5,000 Open Source Mini-Challenge**; record <3:00 YouTube video; write 5 Friction Log entries.	Public GitHub repo with visible MIT license in About section (or private repo shared with `testing@devpost.com`, `chris-trag`, `knmeiss`, `giolaq`, `anishamalde`, `mosesroth`, `emersonsklar`) + **10% Friction Log bonus**.

High-Scoring Friction Log Entries to Include in Submission (Up to +10% Judging Bonus)

- **Friction Entry 1 — Public Docs vs. Partner-Gated CLI & Simulator:** The Alexa+ MCP QuickStart Guide instructs builders to run `alexa-ai configure` and `alexa-ai new mcp` without a top-banner warning that the CLI is partner-gated (only clarified in the Devpost FAQs). *Workaround & Suggestion:* Built `alexa-addon-packager` Agent Skill to generate compliant `addon.json` manifests offline; suggest adding an offline schema validator to public npm.
- **Friction Entry 2 — OAuth 2.1 `401 Unauthorized` Header Divergence:** Alexa+ MCP Authentication requires returning `401 Unauthorized` *without* a `WWW-Authenticate` header, whereas standard RFC 9728 OAuth 2.1 middlewares attach `WWW-Authenticate: Bearer resource_metadata=...` by default. *Workaround:* Added an Alexa+ compatibility middleware flag to strip `WWW-Authenticate` when queried by Alexa+ user agents.
- **Friction Entry 3 — MCP Spec Era Negotiation (`2025-11-25` vs. `2026-07-28`):** Bridging session-based Streamable HTTP (2025-11-25) with stateless header-mirroring clients (2026-07-28) requires conditional inspection of `MCP-Protocol-Version` so neither client era fails with `400 Bad Request`.